

VELLORE INSTITUTE OF TECHNOLOGY BHOPAL

School of Computer Science & Engineering

VITyarthi — Build Your Own Project Evaluation

BANK MANAGEMENT SYSTEM

PROJECT REPORT

A Python Console Application with JSON-Based Persistence

Student Name:	Aditya Raj Kuntal
Registration / Roll No:	26BAI10740
Course Code & Title:	CSE1021 — Introduction to Problem Solving and Programming
Project Title:	Bank Management System
Project Domain:	Python Programming / Management System
Submission Platform:	VITyarthi Learning Portal
Date of Submission:	September 2026

Table of Contents

1. Introduction
2. Problem Statement
3. Objectives
4. Scope of the Project
5. Functional Requirements
6. Non-Functional Requirements
7. Technical Expectations and Technologies
8. System Architecture
9. Workflow Diagram
10. Use Case Diagram
11. Class / Component Design
12. Sequence Diagram
13. Storage Design
14. Design Decisions & Rationale
15. Implementation Details
16. Screenshots / Results
17. Testing Approach
18. Challenges Faced
19. Learnings & Key Takeaways
20. Future Enhancements
21. GitHub Repository & Version Control
22. Conclusion
23. References

1. Introduction

The Bank Management System is a Python-based academic project that simulates the basic operations of a bank through a simple, menu-driven console application. A user can create an account, log in securely with a PIN, check the balance, deposit and withdraw money, transfer money to another account, change the PIN and view the transaction history. The project applies programming concepts to a practical, real-world context, as required by the VITYarthi Build Your Own Project guidelines.

The application is divided into separate modules for account handling, authentication, transactions, transfers, validation and storage. All account information is saved in a local JSON file so that the data remains available after the program is closed and started again. Because the project uses only Python and a local file, it does not depend on any external database server.

2. Problem Statement

Managing customer accounts, balances and transactions manually is slow and error-prone. Records can be entered wrongly, balances may be miscalculated, and it becomes difficult to trace what happened to an account over time. Unauthorised access to an account is another concern when no verification step exists.

The proposed system provides a small computerised solution in which accounts are created and protected by a PIN, money movements are validated before they are applied, every transaction is recorded, and all data is stored persistently. This makes basic banking operations organised, traceable and easy to demonstrate.

3. Objectives

- Identify a practical problem related to basic banking operations and solve it using Python.
- Design and implement a modular application with clearly separated responsibilities.
- Allow users to create accounts and access them only after PIN-based authentication.
- Support balance enquiry, deposit, withdrawal and money transfer between accounts.
- Maintain a transaction history for every account.
- Validate all user input and handle errors without crashing the program.
- Store data in a JSON file so that records persist between program runs.
- Document the design using diagrams and verify behaviour with test cases.

4. Scope of the Project

The project is a local, single-user, console-based academic application. It covers account creation, PIN-based login, balance enquiry, deposits, withdrawals, transfers between accounts registered in the same system, PIN change and transaction history.

It does not attempt to provide real-world banking features such as internet or mobile banking, connection to a payment network, interest calculation, loans, cards, multi-branch support or integration with any real bank. It is intended for learning and demonstration only.

5. Functional Requirements

Module	Input	Output / Function
Account Creation	Name, 4-digit account number, 4-digit PIN and initial deposit	A new account record is created and saved to accounts.json
Authentication	Account number and PIN	Login succeeds only for valid credentials; the user then reaches the account menu
Balance Enquiry	Logged-in account	Current balance of the account is displayed
Deposit	Amount to deposit	Balance is increased and the transaction is recorded
Withdraw	Amount to withdraw	Balance is reduced if funds are sufficient; otherwise the request is rejected
Money Transfer	Receiver account number and amount	Amount is debited from the sender and credited to the receiver; a history entry is added to both accounts
Change PIN	Current PIN and new PIN	PIN is updated and saved after verification
Transaction History	Logged-in account	List of previous transactions of the account is displayed
Data Storage	Account and transaction records	Records are loaded from and saved to the JSON file
Input Validation	Account number, PIN and amounts typed by the user	Account number and PIN must be 4-digit numbers and amounts must not be negative; invalid input is rejected with a message

6. Non-Functional Requirements

Attribute	Description
Usability	A simple numbered menu with clear prompts and messages guides the user through every operation.
Security	Account access is protected by a PIN; operations are available only after successful login.
Reliability	Data is saved after every data-changing operation so that no completed transaction is lost.
Performance	All operations are local and fast for the small data volume expected in an academic demonstration.
Maintainability	Features are split into separate Python modules, making the code easy to read, test and extend.
Error Handling	Invalid amounts, unknown accounts, wrong PINs and insufficient funds are handled without stopping the program.
Portability	The application runs on any system with Python installed and needs no additional server or service.
Resource Efficiency	A single local JSON file is used for storage; no database server or external package is required.

7. Technical Expectations and Technologies

Item	Used in Project
Language	Python 3 (Python 3.14 used for development)
Development Tool	Visual Studio Code
Programming Style	Functions and modules
Storage	Local JSON file (accounts.json)
Interface	Command-line (console) menu
Validation	Custom validation functions in validation.py
Testing	tests.py together with manual functional testing
Version Control	Git and GitHub repository
External Packages	None required to run the application

The project contains more than five meaningful Python modules, including the main program, account handling, authentication, transactions, transfer, validation, storage and test modules.

8. System Architecture

The system follows a simple modular architecture. main.py presents the menu and controls the flow of the program. The feature modules (account, authentication, transactions and transfer) perform the requested operations. The validation module checks user input, and the storage module reads and writes the accounts.json data file. When the program starts, main.py loads all accounts into memory and it saves them again after data-changing operations.

Project Folder Structure

```
Bank-Management-System/  
|-- main.py  
|-- account.py  
|-- authentication.py  
|-- transactions.py  
|-- transfer.py  
|-- validation.py  
|-- storage.py  
|-- tests.py  
|-- accounts.json  
|-- README.md  
|-- statement.md  
`-- .gitignore
```

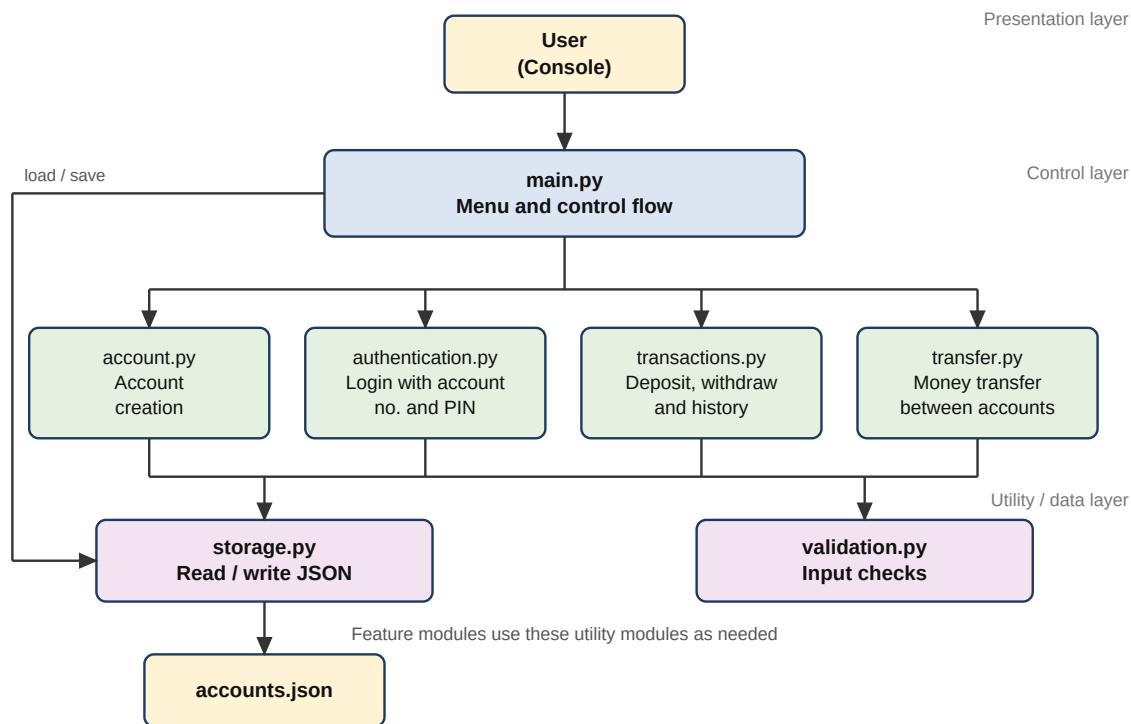


Figure 8.1: High-level system architecture

9. Workflow Diagram

The workflow below shows how a typical session proceeds. The program starts at the main menu, the user creates an account or logs in, and after the PIN is verified the user selects an operation. The input is validated, the operation is performed, and the updated data is saved to the JSON file before the result is shown.

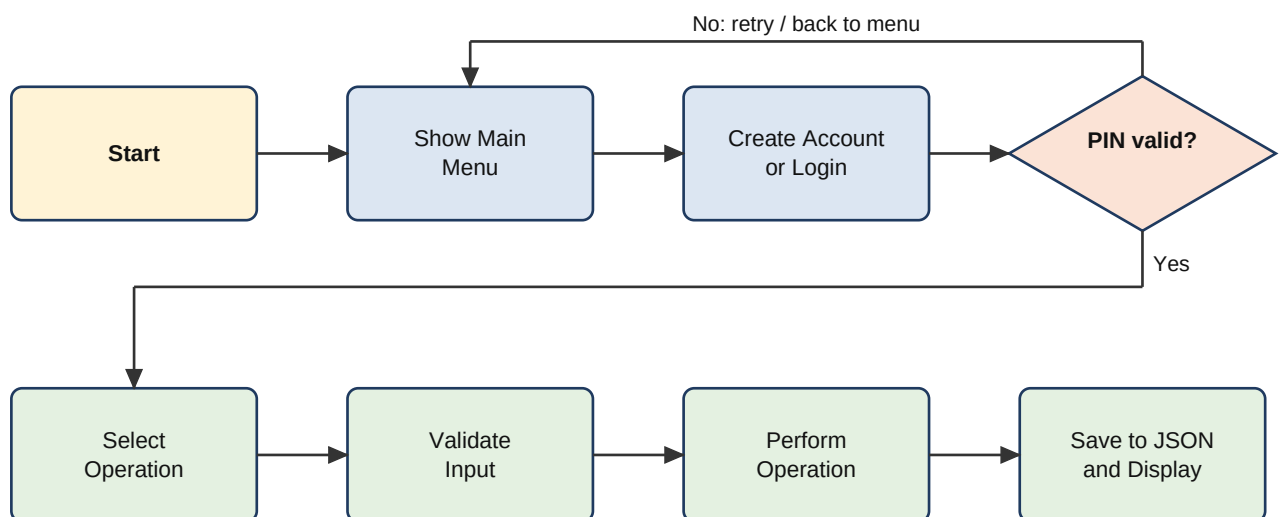


Figure 9.1: Workflow of a banking session

10. Use Case Diagram

The main user of the system is the customer (account holder). The customer interacts with the system to create an account, log in with a PIN, check the balance, deposit, withdraw, transfer money, change the PIN, view the transaction history and log out.

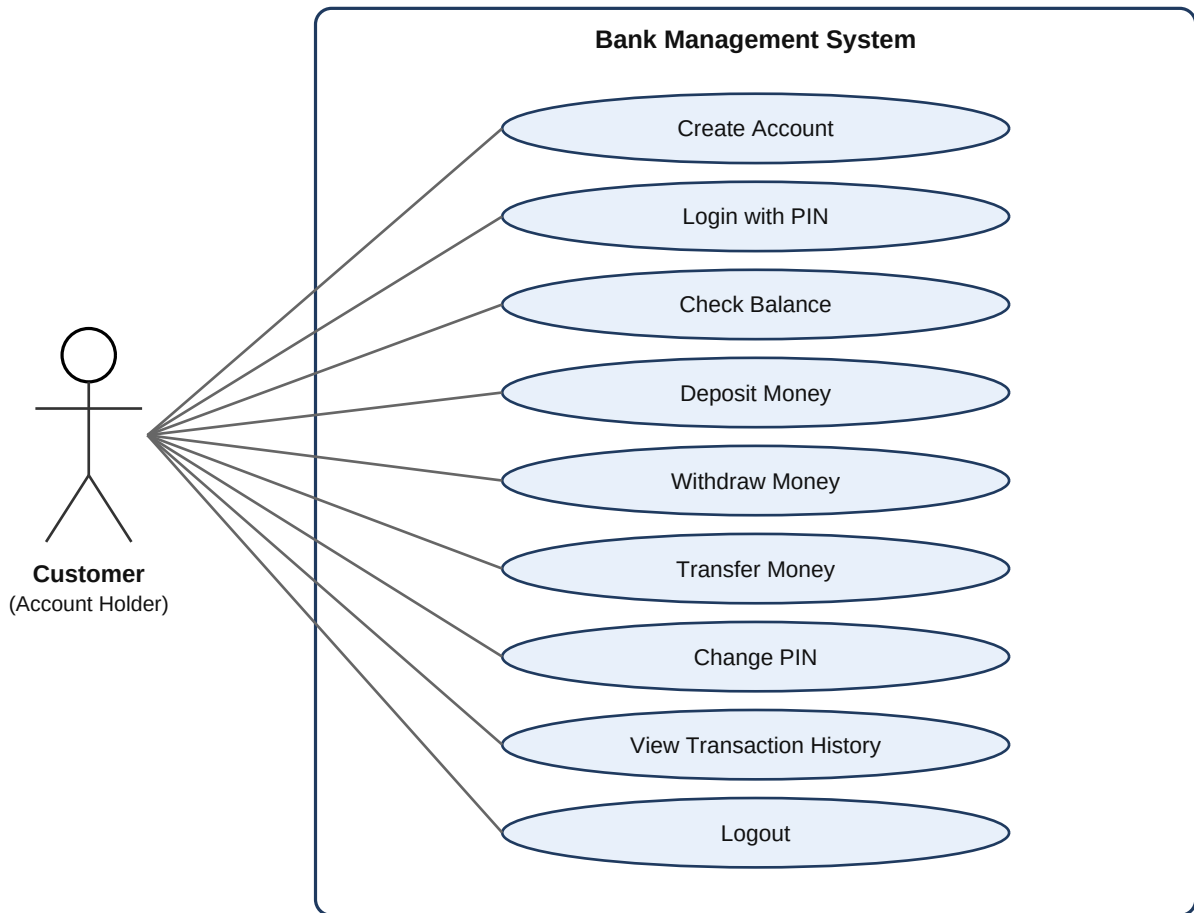


Figure 10.1: Use case diagram

Actor	Use Cases
Customer (Account Holder)	Create account; login with PIN; check balance; deposit money; withdraw money; transfer money; change PIN; view transaction history; logout

11. Class / Component Design

The project is designed around modules, each with one clear responsibility. The component diagram shows how the modules depend on one another, and the table describes the role of each file.

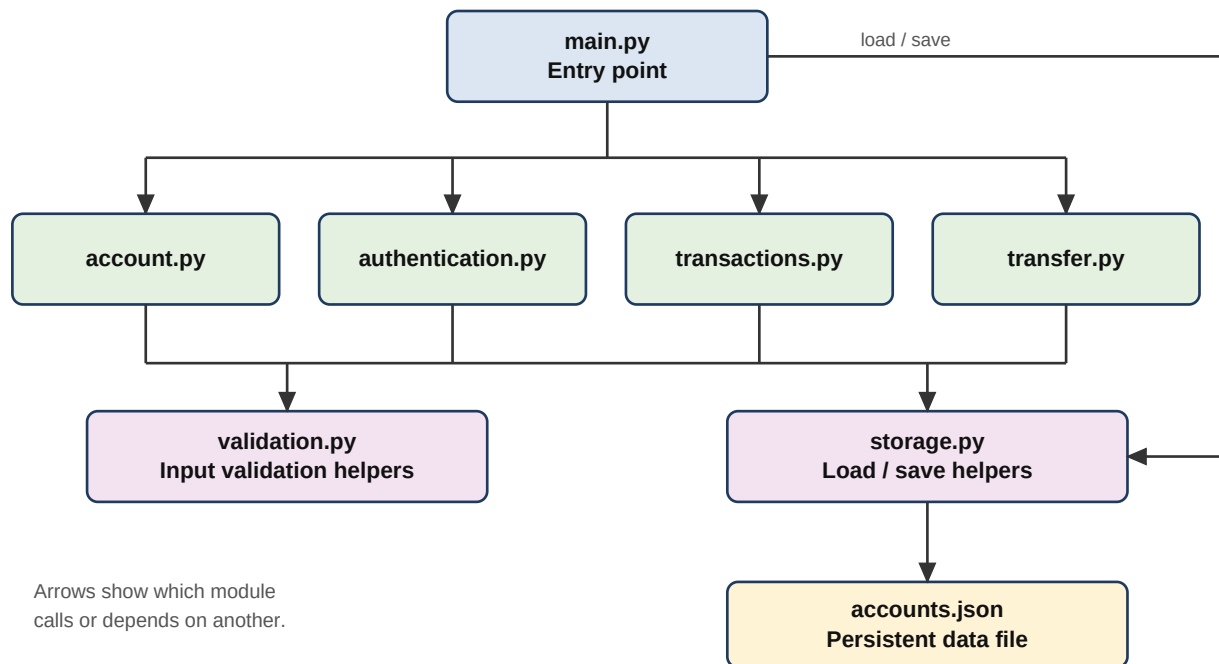


Figure 11.1: Component / module structure

Module	Responsibility
main.py	Entry point of the program; loads the accounts, displays the welcome menu and the account menu, and calls the other modules according to the user's choice.
account.py	Provides create_account(), which creates a new account record.
authentication.py	Provides login(), which verifies the account number and PIN entered by the user.
transactions.py	Provides deposit(), withdraw() and show_transaction_history(); records each transaction in the account history.
transfer.py	Provides transfer_money(), which moves an amount from the logged-in account to another existing account and records it in both histories.
validation.py	Provides validate_account_number(), validate_pin() and validate_amount() as reusable input checks.
storage.py	Provides load_accounts() and save_accounts() to read and write accounts.json.
tests.py	Contains test cases used to verify the behaviour of the modules.

12. Sequence Diagram

The sequence diagram shows the main interactions when a customer logs in and then transfers money to another account. It illustrates how the menu, authentication, transfer and storage modules cooperate: `transfer_money()` checks the receiver and the balance and updates both accounts, and `main.py` then saves the accounts to `accounts.json`.

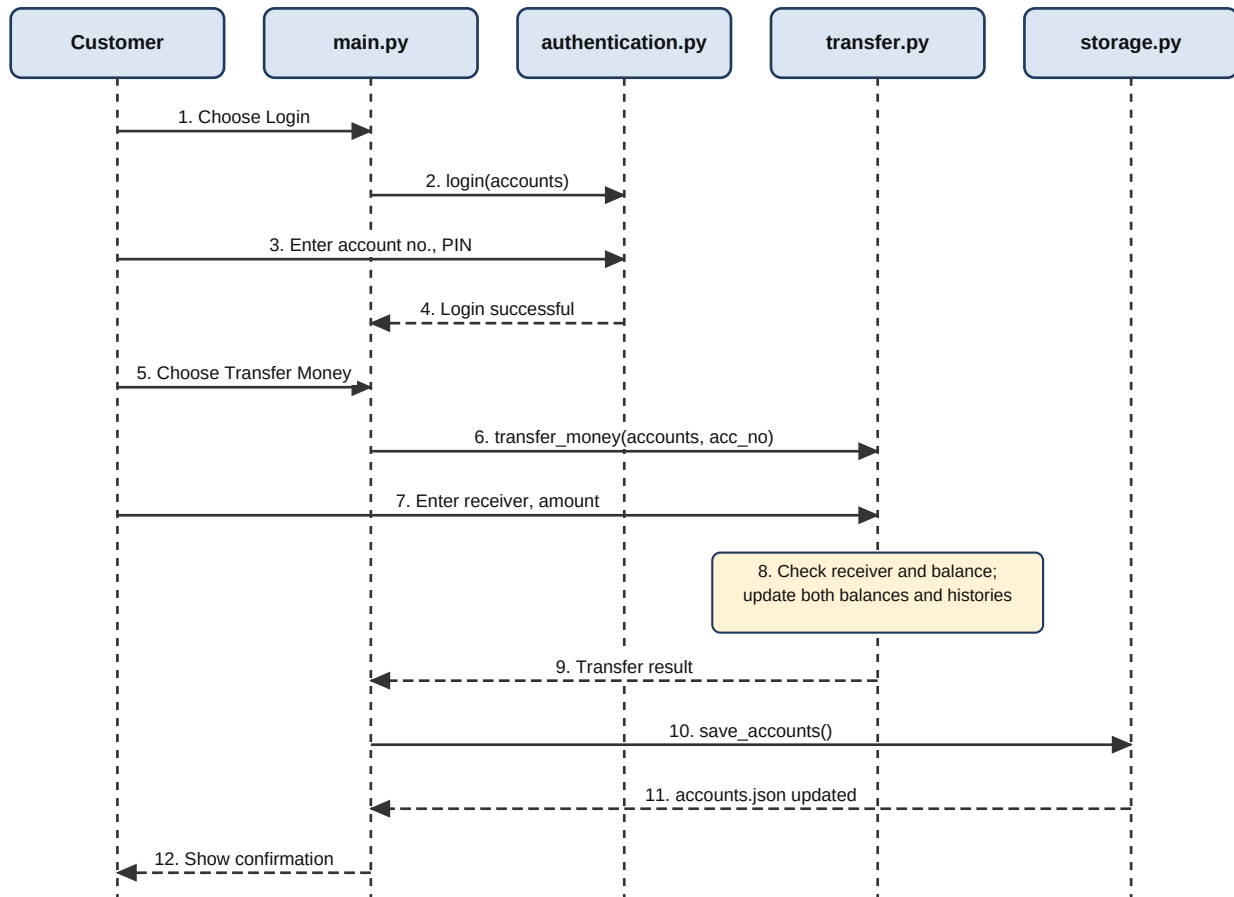


Figure 12.1: Sequence diagram for login and money transfer

13. Storage Design

The project uses local file storage instead of a database server. Account information is kept in a JSON file, which is a human-readable text format supported directly by Python's built-in `json` module. The storage module loads the file when the program starts and writes it back after every data-changing operation.

File	Stored Information
accounts.json	Dictionary of all accounts, keyed by account number

Each account entry contains the following fields:

Field	Type	Description
name	Text	Name of the account holder
pin	Text	4-digit PIN used to log in to the account
balance	Number	Current balance of the account
transactions	List of text	History entries for deposits, withdrawals and transfers sent or received

```
1  {
2      "****": {
3          "name": "Aditya",
4          "pin": "****",
5          "balance": 1101.0,
6          "transactions": [
7              "Deposited \u20b9300.0",
8              "Withdrawn \u20b9199.0"
9          ]
10     }
11 }
```

Figure 13.1: Structure of accounts.json (account number and PIN masked)

For security reasons, the account number and PIN shown in the screenshots of this report are masked.

14. Design Decisions & Rationale

- **Python** was chosen because it clearly demonstrates programming fundamentals and is easy to read and demonstrate.
- **Modular structure** keeps each feature in its own file, which makes the code easier to understand, test and maintain.
- **Console interface** keeps the focus on program logic and data handling rather than on user-interface design.
- **JSON storage** provides persistent, human-readable storage using only Python's built-in functionality.
- **Separate validation module** keeps reusable input checks (account number, PIN and amount) in one place instead of repeating them.
- **PIN-based login** ensures that balance, transfer and history operations are available only to the account holder.

15. Implementation Details

15.1 Main Program

main.py imports load_accounts and save_accounts from storage.py, create_account from account.py, login from authentication.py, deposit, withdraw and show_transaction_history from transactions.py, and transfer_money from transfer.py. At start-up it loads all accounts from accounts.json into a dictionary and then runs a loop that shows the welcome menu: Create Account, Login and Exit. After a successful login, the account menu offers Check Balance, Deposit Money, Withdraw Money, Transfer Money, Change PIN, Transaction History and Logout. Any wrong option prints an Invalid choice message and the menu is shown again.

```
1  from storage import load_accounts, save_accounts
2  from account import create_account
3  from authentication import login
4  from transactions import deposit, withdraw, show_transaction_history
5  from transfer import transfer_money
6
7  accounts = load_accounts()
8
9  while True:
10
11     print("\n=====")
12     print("    WELCOME TO BANK SYSTEM")
13     print("=====")
14     print("1. Create Account")
15     print("2. Login")
16     print("3. Exit")
17
18     choice = input("Enter your choice: ")
19
20     if choice == "1":
21
22         create_account(accounts)
```

Figure 15.1: Imports and main menu loop in main.py

15.2 Account Management

create_account() asks for the account holder's name, a 4-digit account number, a 4-digit PIN and an initial deposit. The new record, holding the name, PIN, balance and an empty transaction list, is added to the accounts dictionary, and main.py then calls save_accounts() so that the account is written to accounts.json.

15.3 Authentication

login() asks for the account number and PIN and compares them with the stored account. When they match, the program prints Login successful and the account menu is opened; otherwise access is denied.

15.4 Transactions

transactions.py contains deposit(), withdraw() and show_transaction_history(). Amounts are checked before they are applied, a withdrawal is rejected when the balance is not sufficient, and every successful operation adds a text entry to the account's transaction list. The history option prints these entries one by

one.

15.5 Money Transfer

`transfer_money(accounts, account_number)` asks for the receiver's account number and rejects the request if that account does not exist or if it is the sender's own account. It then reads the amount as a decimal number and rejects negative amounts, amounts larger than the sender's balance, and values that are not numbers. On success it subtracts the amount from the sender, adds it to the receiver, and adds a history entry to both accounts (a Transferred entry for the sender and a Received entry for the receiver) before printing a success message.

15.6 Validation

`validation.py` contains three small helper functions:

- **`validate_account_number()`** returns True only when the value consists of digits and is exactly 4 characters long.
- **`validate_pin()`** applies the same rule to the PIN, so a PIN must be exactly 4 digits.
- **`validate_amount()`** returns True when the amount is not negative.

```
def validate_account_number(account_number):
    return account_number.isdigit() and len(account_number) == 4

def validate_pin(pin):
    return pin.isdigit() and len(pin) == 4

def validate_amount(amount):
    return amount >= 0
```

15.7 Storage

`storage.py` provides `load_accounts()` and `save_accounts()`. The first reads `accounts.json` when the program starts and the second writes the updated dictionary back to the file, so that account data persists between runs.

16. Screenshots / Results

The following screenshots were taken while running the Bank Management System in the VS Code terminal. They demonstrate account creation, login, balance enquiry, deposit, withdrawal, money transfer and transaction history. The account number and PIN typed during the demonstration are masked for security.

16.1 Main Menu

On start-up the program displays the welcome menu with the options Create Account, Login and Exit.

```
=====
      WELCOME TO BANK SYSTEM
=====
1. Create Account
2. Login
3. Exit
Enter your choice: █
```

Figure 16.1: Main menu

16.2 Account Creation

Choosing option 1 asks for the name, a 4-digit account number, a 4-digit PIN and an initial deposit of 1000.

```
=====
      WELCOME TO BANK SYSTEM
=====
1. Create Account
2. Login
3. Exit
Enter your choice: 1
Enter your name: Aditya
Enter 4-digit account number: ****
Create 4-digit PIN: ****
Enter initial deposit: 1000█
```

Figure 16.2: Account creation

16.3 Login

Choosing option 2 asks for the account number and PIN. After correct credentials the program confirms a successful login.

```
=====
WELCOME TO BANK SYSTEM
=====
1. Create Account
2. Login
3. Exit
Enter your choice: 2
Enter account number: ****
Enter PIN: ****
Login successful!
```

Figure 16.3: Login with account number and PIN

16.4 Account Menu and Balance Enquiry

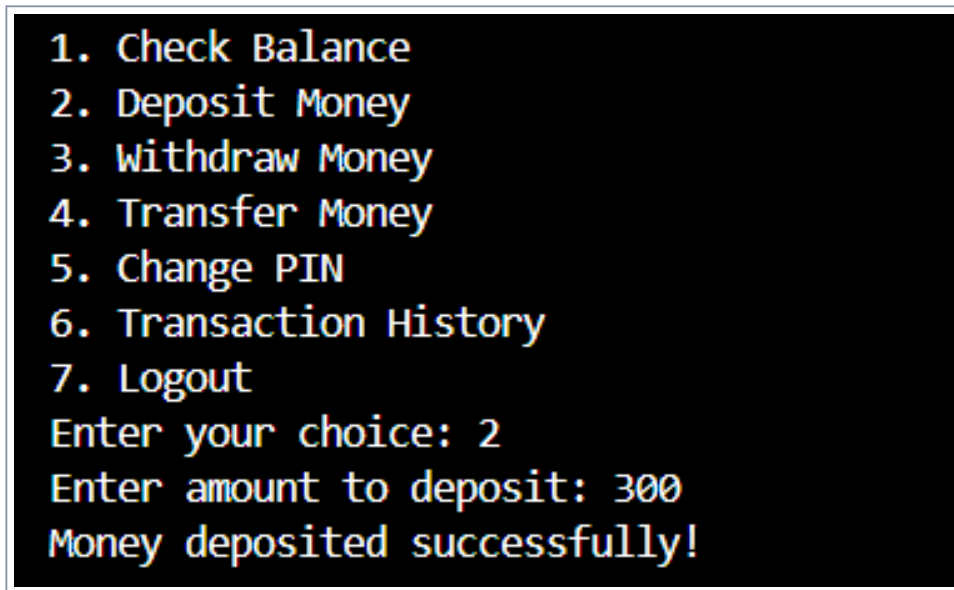
After login the account menu is displayed. Option 1 shows the current balance of the account.

```
----- ACCOUNT MENU -----
1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Transfer Money
5. Change PIN
6. Transaction History
7. Logout
Enter your choice: 1
Current Balance: ₹ 1000.0
```

Figure 16.4: Account menu and balance enquiry

16.5 Deposit

Option 2 asks for an amount and adds it to the balance. Here 300 is deposited successfully.

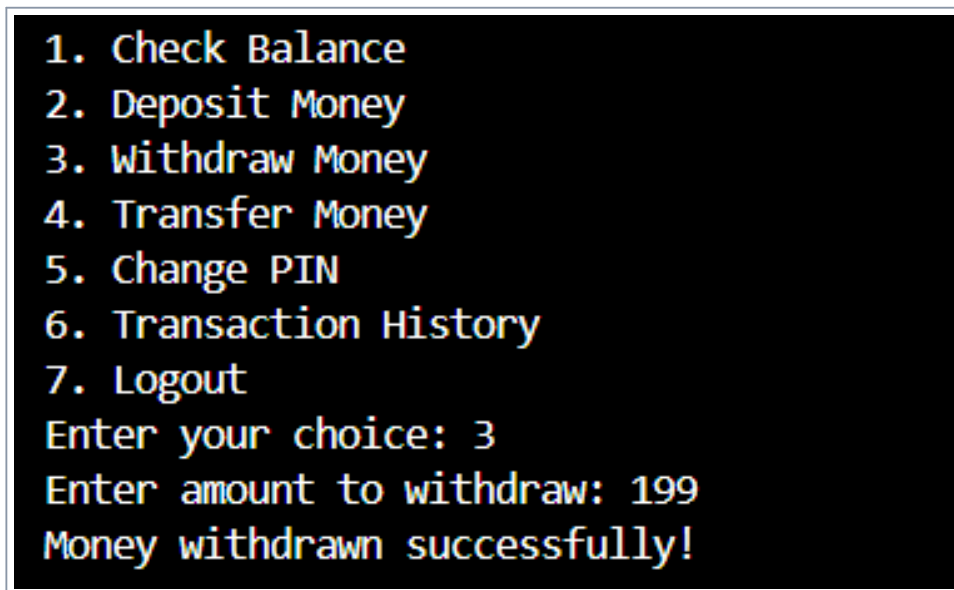


```
1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Transfer Money
5. Change PIN
6. Transaction History
7. Logout
Enter your choice: 2
Enter amount to deposit: 300
Money deposited successfully!
```

Figure 16.5: Depositing money

16.6 Withdraw

Option 3 asks for an amount and deducts it from the balance. Here 199 is withdrawn successfully.



```
1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Transfer Money
5. Change PIN
6. Transaction History
7. Logout
Enter your choice: 3
Enter amount to withdraw: 199
Money withdrawn successfully!
```

Figure 16.6: Withdrawing money

16.7 Money Transfer

Option 4 asks for the receiver's account number and the amount. Here 100 is transferred successfully to another account.

```
2. Deposit Money
3. Withdraw Money
4. Transfer Money
5. Change PIN
6. Transaction History
7. Logout
Enter your choice: 4
Enter receiver account number: ****
Enter amount to transfer: 100
Money transferred successfully!
```

Figure 16.7: Transferring money (receiver account number masked)

16.8 Transfer to a Non-Existent Account

When the receiver account number is not found, the program prints an error message, makes no change to any balance and returns to the account menu.

```
Receiver account does not exist!

----- ACCOUNT MENU -----
1. Check Balance
2. Deposit Money
3. Withdraw Money
4. Transfer Money
5. Change PIN
6. Transaction History
7. Logout
Enter your choice: 7
```

Figure 16.8: Transfer rejected because the receiver account does not exist

16.9 Transaction History

Option 6 lists all transactions recorded for the account, showing the deposit and the withdrawal made above.

```
3. Withdraw Money
4. Transfer Money
5. Change PIN
6. Transaction History
7. Logout
Enter your choice: 6

----- TRANSACTION HISTORY -----
- Deposited ₹300.0
- Withdrawn ₹199.0
```

Figure 16.9: Transaction history

16.10 Summary of Results

Step	Amount	Balance After Step
Initial deposit at account creation	1000.0	1000.0
Deposit	300.0	1300.0
Withdrawal	199.0	1101.0

The balance of 1101.0 after the deposit and withdrawal steps is the same value that is saved in accounts.json (Figure 13.1), which confirms that the data is stored correctly.

17. Testing Approach

The project is verified through the test cases in tests.py and through manual functional testing using the menu options. Each major operation is checked by entering sample data and confirming the displayed result and the saved record.

ID	Test Area	Input / Action	Expected Result
T01	Create account	Valid name, 4-digit account number, 4-digit PIN and initial deposit	New account is created and saved to accounts.json
T02	Invalid account number or PIN	Value that is not exactly 4 digits (letters or wrong length)	Input is rejected and the user is asked again
T03	Login	Correct account number and PIN	Login succeeds and the account menu opens
T04	Login failure	Correct account number, wrong PIN	Login is denied with an error message

ID	Test Area	Input / Action	Expected Result
T05	Check balance	Select balance option	Current balance is displayed correctly
T06	Deposit	Valid positive amount	Balance increases and a transaction is recorded
T07	Invalid amount	Negative or non-numeric amount	Operation is rejected
T08	Withdraw	Amount less than or equal to balance	Balance decreases and a transaction is recorded
T09	Insufficient funds	Withdrawal amount greater than balance	Withdrawal is rejected and balance is unchanged
T10	Transfer	Existing receiver account and valid amount	Sender is debited, receiver is credited, and a history entry is added to both accounts
T11	Unknown receiver	Receiver account number that does not exist	Message that the receiver account does not exist; no balance changes
T12	Transfer to own account	Receiver number equal to own account number	Transfer is rejected with a message
T13	Transfer beyond balance	Amount greater than sender's balance	Insufficient balance message; no balance changes
T14	Change PIN	Correct current PIN and new PIN	PIN is updated and the new PIN works at next login
T15	Transaction history	Select history option	Previous transactions are listed
T16	Persistence	Restart the program after operations	Accounts and balances are still available
T17	Logout	Select logout	User returns to the main menu

18. Challenges Faced

- Keeping the different modules connected while maintaining a clean and simple program structure.
- Ensuring that balances and history stay consistent, especially when a transfer changes two accounts at once.
- Making sure that changes are saved correctly to the JSON file and loaded again on the next run.
- Handling invalid input, such as wrong types of values or negative amounts, without stopping the application.
- Protecting sensitive data such as PINs and account details when documenting and sharing the project.
- Providing enough functionality to meet project requirements without unnecessary complexity.

19. Learnings & Key Takeaways

- Learned how to divide a Python application into multiple cooperating modules.
- Practised file handling and persistent storage using JSON.
- Understood the importance of input validation and basic error handling.
- Learned how authentication protects user data and how to structure a login flow.
- Gained experience in writing test cases and verifying features systematically.
- Learned how diagrams and documentation help explain a software project clearly.
- Practised using Git and GitHub to organise and share a project.

20. Future Enhancements

- Add a graphical user interface.
- Replace the local JSON file with a relational database such as SQLite or MySQL.
- Add stronger security: store PINs in hashed form instead of plain text, lock an account after repeated failed attempts and keep audit logs.
- Add an administrator role for managing accounts.
- Add different account types with interest calculation.
- Generate downloadable statements and transaction receipts.
- Add backup and restore functionality for account data.
- Extend the system for secure multi-user access over a network.

21. GitHub Repository & Version Control

The project is maintained as a public GitHub repository containing the source code, data file, documentation and tests. Git is used to track changes, and the repository is used for submission of the organised project.

Item	Details
Repository	https://github.com/noctadityaa-bit/Bank-Management-System
Branch	main
Source files	main.py, account.py, authentication.py, transactions.py, transfer.py, validation.py, storage.py, tests.py
Data file	accounts.json
Documentation	README.md (overview, features, setup and run steps) and statement.md (problem statement and scope)
Other	.gitignore

22. Conclusion

The Bank Management System demonstrates how Python programming concepts can be combined to solve a practical problem. The project provides account creation, PIN-based authentication, deposits, withdrawals, transfers, transaction history, input validation and persistent JSON storage in a modular console application.

Its modular structure makes the application easy to understand, test and extend. The project is aligned with the VITyarthi requirement to identify a meaningful problem, design a technical solution, implement it using learned concepts and document the resulting system.

23. References

- VITyarthi – Build Your Own Project: General Project Instructions and Submission Guidelines provided for project evaluation.
- Python documentation, including the built-in json module and standard language features used in the implementation.
- Project source code, README.md and statement.md:
<https://github.com/noctadityaa-bit/Bank-Management-System>